利用Docker搭建Linux C++ Boost开发环境

Posted on 2023-07-22 12:46:44 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2Suf5jJjQEVKDClYNgIiKV9ipcb)

简介

本文介绍如何使用Docker搭建Linux环境下C++的开发环境。众所周知C++的开发环境分为Windows和Linux两种，在windows配置C++开发环境并实现了Asio服务器的开发，我们也做过QT的配置和使用，前面的教程已经很完善了，接下来介绍如何在Linux系统部署C++开发环境。

宿主机和Docker

对于Linux系统，选择性很多包括Centos, Redhat, Ubuntu等，大家可以选择在宿主机上直接安装Linux某个版本的系统，也可选择Docker环境部署C++开发环境。
选择Docker的好处如下：
1 可移植性好，不会因为宿主机的差异导致程序移植后无法运行
2 Docker安装的库不会影响宿主机环境，也不会被宿主机环境影响，程序运行环境纯净。
3 共享性好，可以将配置好的容器打包成镜像提交到Docker Hub或者压缩成压缩包供他人使用，提升开发效率。
为了方便给大家演示并提供给大家一个可移植的环境，我选择用Docker配置C++开发环境。

Docker安装

根据不同的Linux系统可以选择不同的方式安装，我的是Ubuntu系统，所以简单列举下我的安装步骤
1 更新系统软件包列表：
执行以下命令以确保您的软件包列表是最新的：

```
1. sudo apt update
```

2 安装依赖项以允许使用 HTTPS 通过 APT 获取 Docker：
执行以下命令安装所需的软件包：

```
1. sudo apt install -y apt-transport-https ca-certificates curl software-properties-common
```

3 添加 Docker GPG 密钥：
执行以下命令以添加 Docker 的官方 GPG 密钥：

```
1. curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/shar
   e/keyrings/docker-archive-keyring.gpg
```

4 添加 Docker APT 软件仓库：
执行以下命令以将 Docker APT 软件仓库添加到系统中：

```
1. echo "deb [arch=amd64 signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://d
   ownload.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.lis
   t.d/docker.list > /dev/null
```

5 更新软件包列表并安装 Docker：
执行以下命令以更新软件包列表，并安装 Docker CE：

```
1. sudo apt update
2. sudo apt install -y docker-ce docker-ce-cli containerd.io
```

6 验证 Docker 安装是否成功：
执行以下命令以验证 Docker 是否已成功安装：

```
1. sudo docker run hello-world
```

如果一切正常，您将看到一个欢迎消息，表示 Docker 已成功安装并正在运行

搭建C++环境

我们基于ubuntu系统的镜像搭建容器，当然你也可以选择其他的系统如centos等，如果小伙伴不喜欢用Docker，可以直接在宿主机操作。

1 启动ubuntu:18.04容器

```
1. docker run -itd --name cppubuntu ubuntu:18.04
```

2 容器内更新源，这样能安装最新的软件包

```
1. apt-get update
```

3 容器内安装gcc/g++工具包

```
1. sudo apt install build-essential
```

4 容器内测试g++版本和命令

```
1. echo '#include <iostream>' > test.cpp
2. echo 'int main() { std::cout << __cplusplus << std::endl; return 0; }' >> test.cpp
3. g++ -std=c++17 test.cpp -o test
4. ./test
```

输出201703L即证明支持C++17

5 容器内安装wget

```
1. apt install wget
```

6 容器内安装boost

6.1 获取源码包

```
1. wget https://boostorg.jfrog.io/artifactory/main/release/1.82.0/source/boost_1_82_0.tar.gz
```

获取速度较慢，我用windows下载好后用filezilla传递到云服务上，再从云服务上执行拷贝操作放入docker

```
1. docker cp  /home/ubuntu/boost_1_82_0.tar.gz  cppubuntu:/test
```

6.2 下载boost需要的库

```
1. apt-get install  python-dev autotools-dev libicu-dev build-essential libbz2-dev libboost-all-dev
```

6.3 解压缩并安装

```
1. tar zxvf boost_1_82_0.tar.gz
2. cd ./boost_1_82_0
3. ./bootstrap.sh --prefix=/usr/
4. ./b2 install
```

6.4 编码测试

```
1.  vim ./boosthello.cpp
```

写如下代码

```
1. #include <iostream>
2. #include <boost/version.hpp>
3. using namespace std;
4. int main() {
5.     cout << "Boost 版本" << BOOST_VERSION << endl;
6.     return 0;
7. }
```

编译运行

```
1. g++ -o boosthello ./boosthello.cpp
```

执行 `./boosthello` 输出

```
1. Boost 版本108200
```

7 容器内安装git

```
1. apt install git
```

8 拉取项目代码

```
1. git clone https://gitee.com/secondtonone1/boostasio-learn.git
```

9 配置git

```
1. git config --global user.name "secondtonone1"
2. git config --global user.email "secondtonone1@163.com"
```

可以验证一下

```
1. git config --global user.name
2. git config --global user.email
```

10 下载cmake

```
1. wget https://github.com/Kitware/CMake/releases/download/v3.27.0/cmake-3.27.0.tar.gz
```

安装必要的ssl库

```
1. apt install libssl-dev
```

解压缩

```
1. tar -zxvf cmake-3.27.0.tar.gz
```

设置引导程序

```
1. ./bootstrap
```

编译

```
1. make
```

安装

```
1. make install
```

测试安装情况

```
1. cmake --version
```

11 编写CMakeLists.txt, 在AsyncServer外层目录

```
cmake_minimum_required(VERSION 3.12)
project(AsyncServer)

# 设置 C++ 标准
set(CMAKE_CXX_STANDARD 17)

# 设置 Boost 的路径
set(BOOST_ROOT /usr)

# 查找 Boost 库的组件
find_package(Boost REQUIRED COMPONENTS system thread)

# 添加可执行文件和源文件
file(GLOB SOURCES ${CMAKE_CURRENT_SOURCE_DIR}/day06-AsyncServer/*.cpp)

add_executable(AsyncServer ${SOURCES})

# 包含头文件路径（包括其他目录）
target_include_directories(AsyncServer
    PRIVATE
    ${CMAKE_CURRENT_SOURCE_DIR}/day06-AsyncServer
    ${CMAKE_CURRENT_SOURCE_DIR}/other_directory
    # 添加其他目录路径...
)

# 包含 Boost 头文件路径
target_include_directories(AsyncServer PRIVATE ${Boost_INCLUDE_DIRS})

# 链接 Boost 库
target_link_libraries(AsyncServer PRIVATE ${Boost_LIBRARIES})
```

新建目录build, 进入build目录，执行 `cmake ..`
生成makefile文件。

在该文件夹执行make 生成可执行文件AsyncServer， 测试可运行。

12 打包容器生成镜像

```
docker commit cppubuntu cppubuntu:1.0
```

将镜像压缩成tar

```
1. docker save -o cppubuntu.tar cppubuntu:1.0
```

传输到百度网盘，大家可以直接使用我的开发环境了。

链接: https://pan.baidu.com/s/1xHP5pSRjFZFiV5qRpgw0KQ?pwd=y468 (https://pan.baidu.com/s/1xHP5pSRjFZFiV5qRpgw0KQ?pwd=y468)

提取码: y468

总结

本文总结了如何使用Docker搭建Linux C++开发环境，读者也可以根据文档提供的步骤在宿主机直接搭建。

浏览(3349) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(571) (JavaScript:void(0);)

热门评论

H B I

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)

博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站

个人微信



个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。

京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)
京ICP备2021033081号 (https://beian.miit.gov.cn/)